

The Good, the Bad, and the Misconfigured: An Empirical Study of Cross-Mechanism Security Policies on the Modern Web

Daphne Pissios, Margaret Potagal
Brandeis University
May 7, 2026

Abstract

Modern web applications rely on multiple browser-enforced mechanisms to isolate content across several origins, including document-level headers (e.g., Cross-Origin Opener Policy), resource-level controls, and embedding restrictions such as iframe sandboxing. While these mechanisms aim to enforce a unified security boundary, they are configured independently and applied inconsistently in practice due to no standard being in place.

This paper presents an empirical study of isolation as a multi-mechanism property across real-world websites. We develop a large-scale crawler to analyze 5,000 websites from the Tranco list, extracting security headers and iframe configurations to evaluate how isolation intent is expressed and enforced.

Our findings reveal widespread inconsistencies, including partial deployments, contradictory configurations, and reliance on incomplete mechanisms. These results suggest that isolation on the web is fragmented, increasing the risk of unintended cross-origin interactions and potential privilege escalation. We conclude with recommendations for improving consistency and propose directions for future research.

1. Introduction

Modern websites integrate resources from multiple origins, including scripts, images, iframes, and third-party services. While this enables rich functionality, it also introduces security risks due to cross-origin interactions. To mitigate these risks, browsers provide multiple isolation mechanisms such as the Cross-Origin Opener Policy (COOP), Cross-Origin Embedder Policy (COEP), Cross-Origin Resource Policy (CORP), Content Security Policy (CSP), and iframe sandboxing.

Despite their shared goal, these mechanisms operate independently and are configured through different channels, including HTTP headers and HTML attributes. As a result, enforcing isolation is not the responsibility of a single mechanism but rather the outcome of their combined use.

This raises several challenges. First, developers must understand and correctly configure multiple mechanisms simultaneously. Second, there is no standard specification guiding how these mechanisms should be composed. Third, inconsistencies across mechanisms may introduce security gaps.

This paper investigates the following research questions: How do websites express isolation intent across multiple mechanisms?

How consistent are these mechanisms within a single site? Are there systematic patterns of incomplete or contradictory configurations?

Contributions

We design and implement a scalable crawler to collect isolation configurations from 5,000 websites from the Tranco list. We define a taxonomy of isolation mechanisms and their roles. We identify and quantify inconsistencies across mechanisms. We provide empirical evidence that isolation is fragmented in real-world deployments.

2. Background

Web isolation mechanisms operate at different layers:

Document-Level Isolation

COOP (Cross-Origin Opener Policy): Controls whether a document shares a browsing context with cross-origin pages.

COEP (Cross-Origin Embedder Policy): Restricts which resources can be embedded unless explicitly allowed.

Resource-Level Isolation

CORP (Cross-Origin Resource Policy): Defines which origins can load a resource.

CSP (Content Security Policy): Controls allowed sources for scripts, frames, and other resources.

Embedding-Level Isolation

X-Frame-Options (XFO): Prevents clickjacking by restricting framing.

iframe sandbox attribute: Restricts capabilities of embedded content (e.g., disabling scripts or forms).

Each mechanism enforces isolation from a different perspective. However, effective security depends on their **consistent combination**.

3. Related Work

Prior work has extensively studied client-side web vulnerabilities, particularly focusing on how attackers exploit inconsistencies in browser-enforced security mechanisms. However, most existing research examines vulnerabilities in isolation, rather than analyzing how multiple protection mechanisms interact. This section reviews three representative studies that highlight weaknesses in client-side security and motivate the need for a compositional analysis of isolation.

The study “The Great Request Robbery: An Empirical Study of Client-side Request Hijacking Vulnerabilities on the Web” investigates a modern class of request forgery attacks that originate from client-side logic. Unlike traditional server-side request forgery, these attacks exploit input validation flaws in JavaScript to hijack outgoing requests. The authors systematize 10 variants of request hijacking vulnerabilities and develop a detection tool to analyze their prevalence at scale. Their findings show that approximately 9.6% of the top 10,000 websites are vulnerable, enabling attacks such as cross-site request forgery, data exfiltration, and arbitrary request manipulation. Importantly, the paper evaluates defenses such as Content Security Policy (CSP), Cross-Origin Opener Policy (COOP), and Cross-Origin Embedder Policy (COEP), concluding that each mechanism only mitigates a subset of attacks. This highlights a key limitation: existing

protections are fragmented and insufficient when deployed independently.

Similarly, “Busting Frame Busting: A Study of Clickjacking Vulnerabilities on Popular Sites” examines the effectiveness of defenses against clickjacking attacks, which rely on embedding a target page within an iframe to trick users into unintended interactions. The study analyzes frame-busting techniques across popular websites and demonstrates that all observed defenses can be bypassed, often using simple or browser-specific techniques. The authors further show that client-side defenses are unreliable and advocate for stronger browser-enforced mechanisms such as X-Frame-Options. This work is particularly relevant to our study because it exposes weaknesses in embedding-level isolation controls, reinforcing the idea that relying on a single mechanism (e.g., iframe restrictions) is insufficient for robust security.

In a related line of work, “Don’t Trust The Locals: Investigating the Prevalence of Persistent Client-Side Cross-Site Scripting in the Wild” explores vulnerabilities arising from improper handling of client-side storage. The authors introduce the concept of persistent client-side XSS, where malicious data stored in mechanisms such as localStorage or cookies is later executed in the browser. Through a large-scale measurement of the Alexa Top 5,000 sites, they find that over 8% of sites exhibit vulnerable data flows, and up to 21% of storage-using sites are susceptible, with a majority of these vulnerabilities being directly exploitable. This work demonstrates that developers often implicitly trust client-side data, leading to exploitable inconsistencies between storage, processing, and execution contexts.

Collectively, these studies reveal a common theme: web security mechanisms are frequently deployed in isolation, inconsistently configured, or incorrectly trusted. While prior work has successfully identified vulnerabilities

in specific mechanisms, such as request handling, iframe embedding, or client-side storage, there is limited understanding of how these mechanisms interact when combined within a single web application.

This paper builds on prior work by shifting the focus from individual vulnerabilities to isolation as a compositional property. Instead of analyzing a single mechanism in isolation, we examine how multiple mechanisms (i.e COOP, COEP, CORP, CSP, and iframe sandboxing) are jointly deployed across real-world websites. Our work complements existing studies by providing a holistic view of isolation, identifying inconsistencies across mechanisms, and quantifying how fragmented deployments may introduce new security risks.

4. Methodology

4.1 Dataset Construction

We construct a dataset of 5,000 websites using a subset of the Tranco top-1M list. The first 5,000 domains are extracted and normalized into valid HTTPS URLs. Each domain is pre-checked for reachability using HTTP HEAD requests with a timeout threshold of 5 seconds. Domains returning server-side errors or failing to respond were classified as unreachable and excluded from further crawling

4.2 Crawling Infrastructure

We implement a Selenium-based crawler using a headless Chromium browser. The crawler is configured with performance logging enabled to capture low-level network events, including HTTP response headers. To improve stability and efficiency, several optimizations are applied, including:

1. Preventing images from loading

2. Disabling browser extensions
3. Capping the JavaScript memory heap at 512 MB
4. Running the browser in Headless mode

The crawler performs the following steps:

1. Load the main page of each site after a fixed wait period (10 seconds)..
2. Capture HTTP response headers using Chrome performance logs.
3. Parse the DOM using BeautifulSoup.
4. Identify iframe elements and extract relevant attributes.
5. Detect login and registration pages with URL matching and analyze them separately when present.

The crawler is designed for robustness, including:

- Automatic browser restarts every 500 websites to prevent memory bloat
- Session recovery and retry logic for browser crashes
- Timeout detection with consecutive failures triggering restarts
- URL sanitization and reachability filtering prior to crawling

4.3 Extracted Features

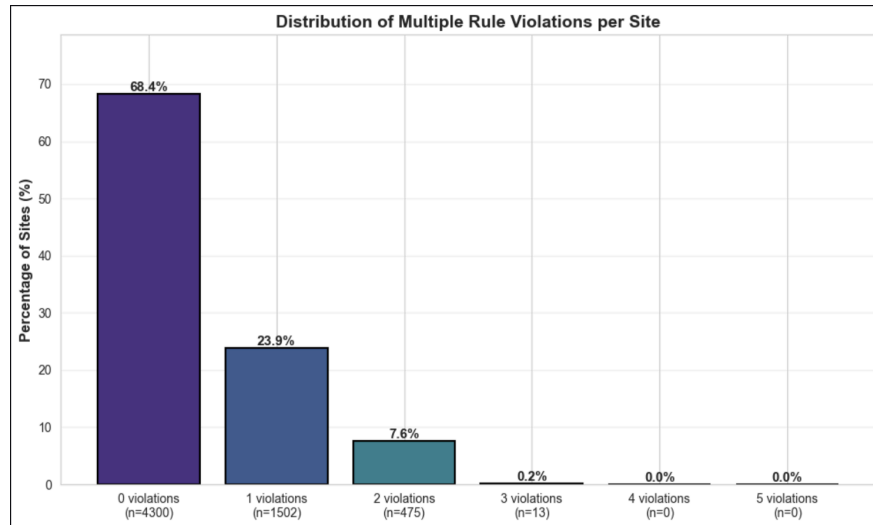
For each analyzed page (main, login, and registration where available), we record:

- Presence and values of security headers (COOP, COEP, CORP, CSP, XFO)
- Number of iframes
- Number of cross-origin iframes
- Number of sandboxed iframes

4.4 Isolation Rules

We define inconsistency patterns based on expected best practices:

- **Rule 1:** Use of iframe sandboxing mechanisms in the absence of COOP/COEP enforcement
- **Rule 2:** COEP enabled while permitting cross-origin iframes without CORP protection
- **Rule 3:** Presence of cross-origin iframes without corresponding sandboxing restrictions
- **Rule 4:** Absence of isolation headers combined with extensive cross-origin embedding
- **Rule 5:** Conflicting security configurations, characterized by restrictive CSP/XFO policies alongside permissive iframe handling



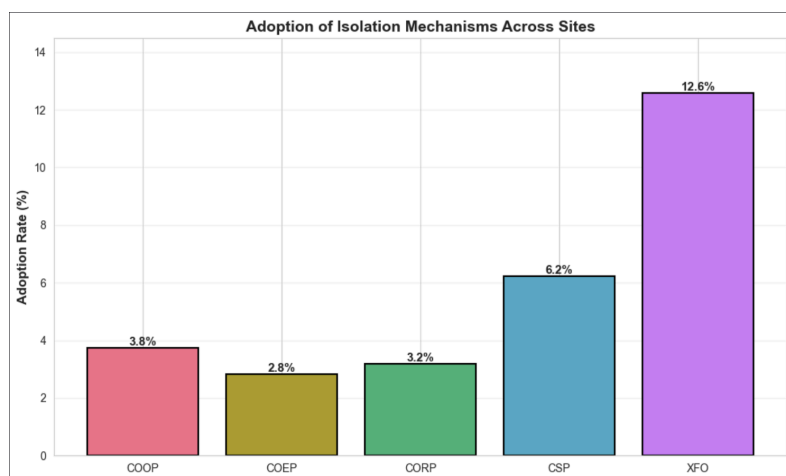
5. Results and Evaluation

Figure 1

5.1 Distribution of Rule Violations Across Sites

The first visualization, Figure 1, shows the distribution of how many isolation rule violations each site exhibits. A clear majority of websites, 68.4% ($n = 4300$), do not violate any of the predefined rules, suggesting that a baseline level of isolation consistency exists across much of the web. However, a significant portion of sites exhibit at least one violation, with 23.9% ($n = 1502$) having exactly one violation and 7.6% ($n = 475$) having two violations. While only a small fraction of sites have three or more violations (0.2%), the presence of any violation indicates gaps in how isolation mechanisms are composed. The long tail of violations, though small, is important because even a single misconfiguration can introduce exploitable behavior. These results suggest that while many sites avoid obvious misconfigurations, a non-trivial percentage still fail to consistently enforce isolation policies. Overall, this distribution highlights that inconsistencies are widespread enough to merit concern, even if severe multi-violation cases are rare.

Figure 2



5.2 Adoption of Isolation Mechanisms

The second visualization (figure 2) presents the adoption rates of key isolation mechanisms across the dataset. Among the mechanisms analyzed, X-Frame-Options (12.6%) has the highest adoption rate, followed by Content Security Policy (6.2%), while more modern headers such as COOP (3.8%), CORP (3.2%), and COEP (2.8%) are significantly less common. This indicates that legacy protections remain more widely deployed than newer, more comprehensive isolation mechanisms. The relatively low adoption of COOP and COEP is particularly notable given their importance for enforcing cross-origin isolation in modern browsers. These results suggest that developers may either be unaware of these newer mechanisms or hesitant to deploy them due to compatibility concerns. Additionally, the gap between CSP and COOP/COEP adoption implies that security is often implemented incrementally rather than holistically. Overall, the visualization reinforces the idea that isolation mechanisms are unevenly adopted, contributing to fragmented security configurations.

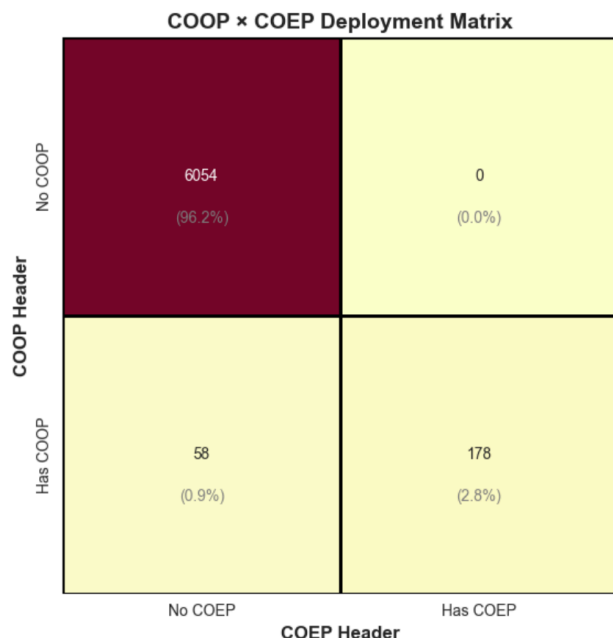


Figure 3

5.3 COOP × COEP Deployment Matrix

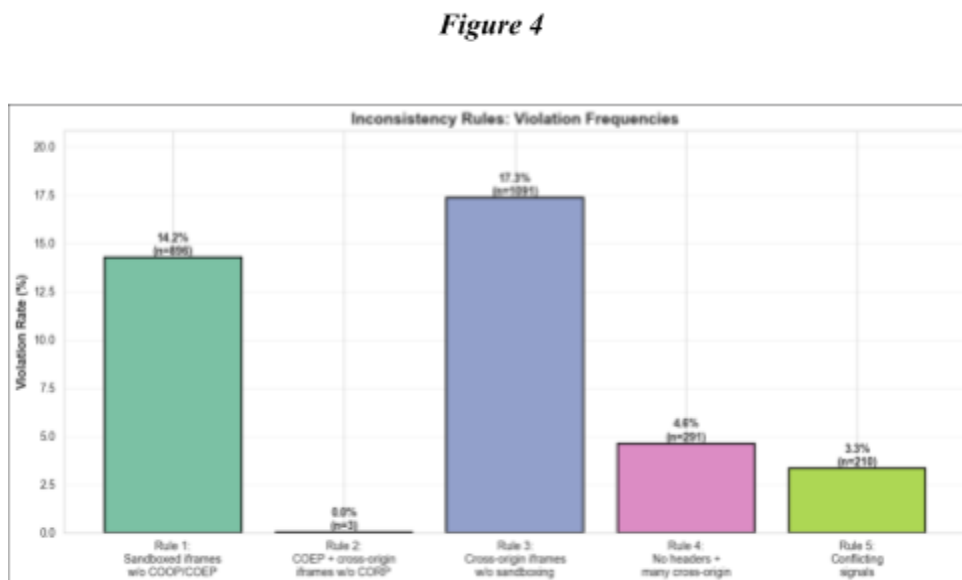
The third visualization provides a joint analysis of COOP and COEP deployment across all sites. The overwhelming majority of websites, 96.2% (n = 6054), deploy neither COOP nor COEP, indicating that full cross-origin isolation is extremely rare in practice. Only 2.8% (n = 178)

of sites deploy both headers, representing those that achieve strong isolation guarantees. A small fraction of sites (0.9%, n = 58) deploy COOP without COEP, while no sites were observed deploying COEP without COOP. This asymmetry suggests that COEP is rarely used independently and is typically only adopted alongside COOP. The near absence of these headers across the dataset highlights a major gap between recommended best practices and real-world deployment. These findings strongly support the argument that modern isolation mechanisms are underutilized. As a result, most websites do not benefit from the stronger protections enabled by full cross-origin isolation.

5.4 Inconsistency Rule Violations

Figure 4 examines the frequency of specific inconsistency patterns across websites. The most common violation is cross-origin iframes without sandboxing (17.3%, n = 1091), indicating that many sites embed external content without applying strict containment controls. This is followed by sandboxed iframes

without COOP/COEP (14.2%, n = 896), suggesting partial isolation where embedding restrictions are used without reinforcing document-level protections. Less frequent but still notable are sites with no headers but many cross-origin iframes (4.6%) and



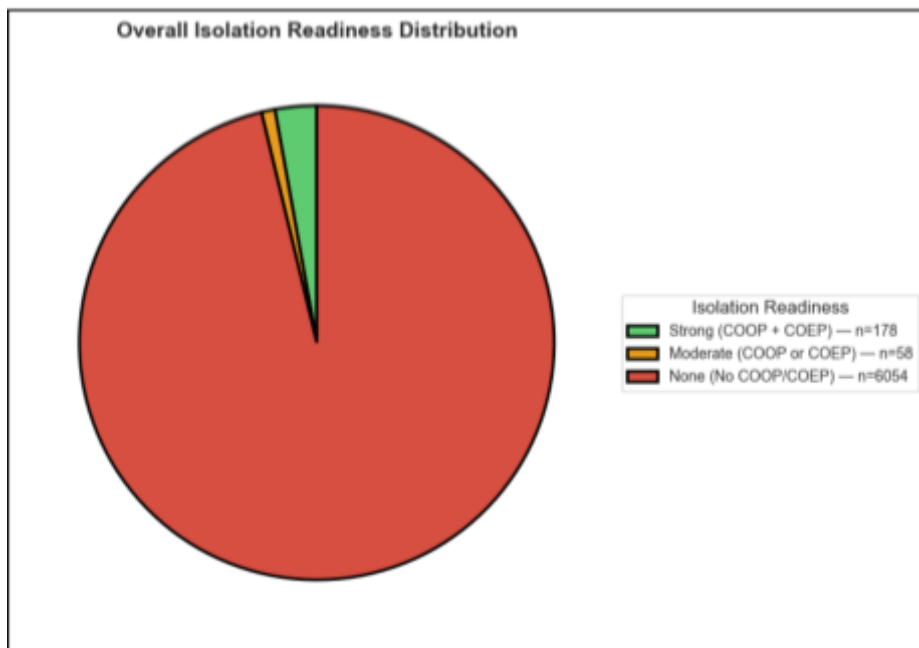
conflicting security signals (3.3%), both of which indicate deeper misconfigurations. Interestingly, no sites were observed violating the rule related to COEP with missing CORP headers, suggesting that this specific configuration is either rare or consistently handled when COEP is present. These results reveal that the most common issues arise from improper handling of embedded content rather than incorrect header usage alone. Overall, the visualization demonstrates that inconsistencies tend to cluster around iframe-related security decisions.

achieves strong isolation ($n = 178$) by deploying both headers, while an even smaller group falls into the moderate category ($n = 58$) with partial deployment. This distribution highlights a stark imbalance in isolation readiness across the web. The dominance of the “no isolation” category suggests that most sites rely on weaker or incomplete security mechanisms. Even among sites that attempt to implement isolation, full compliance with best practices remains rare. These findings emphasize that strong isolation is not yet the norm, leaving many sites potentially exposed to cross-origin risks.

5.5 Overall Isolation Readiness

5.6 Summary of Findings

Figure 5



Across all visualizations, the results consistently show that web isolation is fragmented and inconsistently enforced. While many sites avoid obvious violations, the vast majority fail to adopt modern isolation mechanisms such as COOP and COEP. This gap between partial protections and full isolation creates opportunities for security weaknesses, highlighting the need for more unified and systematic deployment practices.

The final visualization summarizes overall isolation readiness by categorizing sites into strong, moderate, and no isolation. The vast majority of websites fall into the “no isolation” category ($n = 6054$), meaning they deploy neither COOP nor COEP. Only a small subset

6. Discussion and Limitations

6.1 Limitations

- Our crawler analyzes only static snapshots of pages after a fixed wait period and may miss dynamically loaded content.
- We do not execute all JavaScript interactions, limiting visibility into deeper runtime behavior and authenticated states.
- The dataset is limited to 5,000 sites from the Tranco top-1M and may not fully represent the entire web.
- Login and registration pages are detected via URL pattern matching and may be missed or misidentified

6.2 Implications

- Developers struggle to correctly compose multiple isolation mechanisms across different contexts.
- Security policies are often applied incrementally rather than systematically, which leads to partial or inconsistent deployments.
- Fragmented isolation across headers and iframe configurations might increase the attack surface for cross-origin exploits.

6.3 Practical Impact

This work is valuable for:

- Web developers seeking to improve security configurations across both main and sub pages
- Security researchers studying cross-origin vulnerabilities and misconfigurations in practice
- Browser vendors designing more unified isolation models

7. Conclusion and Future Work

This paper presents an empirical study of web isolation as a multi-mechanism property. By analyzing 5,000 websites from the Tranco top-1M list, we demonstrate that isolation is frequently fragmented across HTTP headers and iframe-based controls. Many sites exhibit incomplete or inconsistent configurations across main, login, and registration pages, which may weaken their overall security posture.

Future work could extend this study by:

- Scaling to larger datasets (e.g., the full Tranco list)
- Incorporating dynamic analysis of JavaScript behavior and user interactions
- Building automated tools to detect and fix isolation inconsistencies
- Developing a unified framework for specifying and validating isolation policies
- Improving Detection of sub pages past URL matching

Ultimately, improving web isolation requires not only better mechanisms but also better guidance and tools to help developers to compose them effectively and consistently.

References

- A research-oriented top sites ranking hardened against manipulation—Tranco.* (n.d.). Retrieved May 5, 2026, from <https://tranco-list.eu/>
- Beautiful Soup Documentation—Beautiful Soup 4.14.3 documentation.* (n.d.). Retrieved May 5, 2026, from

- <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>
- Casper da Costa-Luis, Stephen Karl Larroque, Kyle Altendorf, Hadrien Mary, richardsheridan, Mikhail Korobov, Noam Raphael, Ivan Ivanov, Marcel Bargull, Nishant Rodrigues, Shawn, Mikhail Dektiarev, Michał Górny, mjestevens777, Matthew D. Pagel, Martin Zugnoni, CrazyPython, Charles Newey, Antony Lee, ... Hugo van Kemenade. (2026). *tqdm: A fast, Extensible Progress Bar for Python and CLI* (Version v4.67.3) [Computer software]. Zenodo.
<https://doi.org/10.5281/ZENODO.595120>
- Chrome DevTools Protocol*. (n.d.). Retrieved May 5, 2026, from <https://chromedevtools.github.io/devtools-protocol/>
- Chrome specific functionality*. (n.d.). Selenium. Retrieved May 5, 2026, from <https://www.selenium.dev/documentation/webdriver/browsers/chrome/>
- Content Security Policy (CSP)—HTTP* | MDN. (2026, March 22). MDN Web Docs.
<https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP>
- Cross-Origin-Embedder-Policy (COEP) header—HTTP* | MDN. (2026, March 6). MDN Web Docs.
<https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Cross-Origin-Embedder-Policy>
- Cross-Origin-Opener-Policy (COOP) header—HTTP* | MDN. (2025, November 21). MDN Web Docs.
<https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Cross-Origin-Opener-Policy>
- Cross-Origin-Resource-Policy (CORP) header—HTTP* | MDN. (2025, November 21). MDN Web Docs.
<https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Cross-Origin-Resource-Policy>
- HTTP headers—HTTP* | MDN. (2025, December 21). MDN Web Docs.
<https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers>
- Khodayari, S., Barber, T., & Pellegrino, G. (2024). The Great Request Robbery: An Empirical Study of Client-side Request Hijacking Vulnerabilities on the Web. *2024 IEEE Symposium on Security and Privacy (SP)*, 166–184.
<https://doi.org/10.1109/SP54263.2024.00098>
- Performance log* | ChromeDriver. (n.d.). Chrome for Developers. Retrieved May 5, 2026, from <https://developer.chrome.com/docs/chromedriver/logging/performance-log>
- Requests: HTTP for Humans™—Requests 2.34.0.dev1 documentation*. (n.d.). Retrieved May 5, 2026, from <https://docs.python-requests.org/en/latest/>
- re—Regular expression operations*. (n.d.). Python Documentation. Retrieved May 5, 2026, from <https://docs.python.org/3/library/re.html>
- Rydstedt, G., Bursztein, E., Boneh, D., & Jackson, C. (n.d.). *BustingFrameBusting: a Study of Clickjacking Vulnerabilities on Popular Sites*.
- Steffens, M., Rossow, C., Johns, M., & Stock, B. (2019). Don't Trust The Locals: Investigating the Prevalence of Persistent Client-Side Cross-Site Scripting in the Wild. *Proceedings 2019 Network and Distributed System Security Symposium*. Network and

Distributed System Security
Symposium, San Diego, CA.
<https://doi.org/10.14722/ndss.2019.23009>

urllib.parse—Parse URLs into components.
(n.d.). Python Documentation.
Retrieved May 5, 2026, from
<https://docs.python.org/3/library/urllib.parse.html>

WebDriver. (n.d.). Selenium. Retrieved
May 5, 2026, from
<https://www.selenium.dev/documentation/webdriver/>

webdriver-manager: Library provides the
way to automatically manage drivers
for different browsers (Version 4.0.2).
(n.d.). [Python; MacOS, Microsoft ::
Windows, POSIX, Unix]. Retrieved
May 5, 2026, from
https://github.com/SergeyPirogov/webdriver_manager

What is ChromeDriver? (n.d.). Chrome for
Developers. Retrieved May 5, 2026,
from
<https://developer.chrome.com/docs/chromedriver>

X-Frame-Options header—HTTP | MDN.
(2026, March 22). MDN Web Docs.
<https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/X-Frame-Options>